

Image Copy Detection for E-commerce Websites

Technical Overview, Pipeline Architecture, and Multi-Query Experimental Results (Final)

Saketh Pabbu Nerella Venkata Sriram

June 1, 2026

Abstract

E-commerce platforms are increasingly targeted by fraudulent sellers who replicate and modify product listings using stolen images. This report details a deep learning framework developed to detect image copies under various geometric, colorimetric, and noise-based transformations. We describe the dataset challenges, formulate a Siamese ConvNeXt-B network architecture, introduce an epoch-end active harvest loop, and provide a mathematical foundation for robustness guarantees. Finally, we evaluate the system across four benchmark datasets (CIFAR, Flickr, UCID, and Amazon) subjected to 26 simulated attacks. The results show high detection accuracy ($> 91\%$ average precision on CIFAR, $> 89\%$ on Flickr, and $> 86\%$ on UCID) using calibrated thresholds, exposing dataset-specific vulnerabilities, especially for low-contrast and heavily compressed e-commerce images.

Contents

1	The Problem	2
2	The Data Challenge	2
3	The Proposed System Architecture	4
4	Layer I: Dataset Preparation and Pairing Strategy	6
5	Layer II: Siamese Training and Loss Formulations	8
6	Layer III: Indexing and Graph Construction	8
7	Layer IV: Query and Detection Phase	10
8	Future Scope and Project Status	11
9	Experimental Evaluation	12
10	Compact Combined Accuracy Tables	13
11	System Configuration and Technical File Paths	16
12	Conclusion & Future Work	17

1 The Problem

1.1 Background and Motivation

Modern e-commerce platforms (such as Amazon, Flipkart, Alibaba, and Google Shopping) are flooded with duplicate product listings. Fraudulent sellers routinely download high-quality product images from genuine sellers, apply minor visual alterations, and upload them to launch counterfeit, lower-quality listings. This duplicate behavior causes multi-faceted issues:

- **For Customers:** Buyers are misled by authentic-looking product images only to receive cheap knockoffs, eroding trust in online marketplaces.
- **For Original Sellers and Brands:** Creative and financial investments in professional product photography and branding are stolen, leading to lost sales, damaged brand reputation, and expensive, slow legal disputes.
- **For Platforms:** Marketplaces face escalating customer complaints, increased support overhead, and reputational damage.

1.2 Scale of the Challenge

With platforms like Amazon hosting over 350 million active listings and adding thousands of new items daily, manual checking by human moderators is completely infeasible. An automated, robust, and scalable image copy detection system is the only viable solution.

1.3 Defining an Image “Copy”

In this project, an image is defined as a *copy* if it is derived from an original image through modifications that do not alter the core product identity. Common modifications include:

1. **Rotation:** Rotating the product (e.g., shoe facing left vs. right).
2. **Cropping:** Removing background pixels to focus tightly on a sub-feature (e.g., cropping a handbag).
3. **Color Changes and Filters:** Altering brightness, contrast, hue, or applying digital filters.
4. **Flipping (Mirroring):** Horizontally or vertically mirroring the image.
5. **Resizing and Compression:** Lowering the resolution or applying lossy JPEG compression.
6. **Watermarks/Text Overlays:** Adding logos, “SALE” banners, or price text to obscure the background.

Our objective is to train a representation model that is invariant to these transformations, matching any modified duplicate back to its original source.

2 The Data Challenge

2.1 Accessing Web-Scale Data

Acquiring training and evaluation data presents several options and trade-offs:

- **Web Scraping:** Automated scrapers can collect real-world listings, but face IP blocking and legal restrictions under platforms’ Terms of Service.

- **Official APIs:** Some platforms offer structured data access (e.g., Amazon Product Advertising API), but impose rate limits and query constraints.
- **Public Datasets:** Curated public datasets offer stable benchmarks, standard testing ground truths, and reproducible evaluations.

2.2 Differentiating Originals from Copies

In real-world databases, identifying the original creator of an image is mathematically and logically challenging. Practical heuristics include:

- **Timestamp Analysis:** Assuming the earliest uploaded version is the original.
- **Brand Verification:** Trusting images uploaded by verified official brand stores (e.g., Apple, Nike) as originals.
- **Quality Cues:** Assuming higher-resolution, uncompressed files with intact metadata (EXIF data) are the originals.

2.3 Benchmark Datasets Utilized

To evaluate the robustness of our architecture, we utilize four distinct datasets representing diverse visual domains:

1. **CIFAR:** Low-resolution tiny images (32×32) used to stress-test the model's performance on highly degraded inputs.
2. **Flickr:** Real-world photographic images containing diverse outdoor/indoor scenes, people, and backgrounds.
3. **UCID (Uncompressed Colour Image Database):** Used as a standard near-duplicate image benchmark with high-fidelity color profiles.
4. **Amazon:** A subset of Amazon product images capturing various categories, lighting conditions, and typical e-commerce backgrounds.

2.4 Custom E-Commerce Dataset and Categorization Uniqueness

Unlike standard benchmark datasets (such as CIFAR, Flickr, and UCID) that are widely used in general copy detection research, our primary focus is a custom scraped e-commerce product image dataset. This custom dataset is not used in any of the prior research papers, making our evaluation highly realistic, practical, and uniquely tailored to real-world e-commerce scenarios.

The raw dataset was compiled by scraping image URLs and metadata from the official 2023 Amazon Metadata repository. To represent the primary e-commerce product domains, we developed a custom hierarchical taxonomy grouping the scraped listings into 5 distinct product categories. This custom categorization provides a standardized, balanced, and structurally distinct evaluation environment, avoiding category imbalance and representing real e-commerce platform inventory. The raw data metadata sources from the official 2023 Amazon Metadata repository are linked below:

- **Fashion & Apparel:** Clothing, Shoes and Jewelry and Amazon Fashion
- **Electronics:** Electronics and Cell Phones and Accessories
- **Home & Lifestyle:** Home and Kitchen
- **Sports:** Sports and Outdoors

- **Beauty:** Beauty and Personal Care

Detailed characteristics of these datasets are summarized in Table 1.

Table 1: Visual Characteristics and Domain Roles of Evaluation Datasets

Dataset	Image Type	Main Visual Content	Copy Detection Im- plication	Final Store Use
CIFAR	Small natural-object classification images	Low-resolution (32×32) objects such as vehicles, animals, birds, etc.	Compact shape makes global transformations stable, but fine textures are limited.	Benchmark with 17,000 known / 9,000 unknown.
Flickr	Real-world photography	Diverse outdoor/indoor scenes, animals, back-grounds, and buildings.	High visual variety and distractors make crop and background attacks harder.	Benchmark with 10,000 known / 5,000 unknown.
UCID	Content-based re-trieval style images	Varied natural scenes and objects in a smaller, controlled environment.	Controlled copy-detection geometry; store size is naturally limited due to pool size.	Benchmark with relaxed-refill: 4,500 known / 850 unknown.
Amazon	E-commerce product listings	Products across electronics, home, sports, cosmetics, baby, and clothing.	Standard white back-grounds, seller variations, repeated layouts, and category imbalance.	Primary product target: 35,000 known / 15,000 unknown.

3 The Proposed System Architecture

3.1 Pipeline Stages

Our proposed pipeline is divided into seven major structural stages:

1. **Dataset Split:** Source images are partitioned into known (original-bank) and unknown (non-duplicate) subsets.
2. **Pair Manifest:** Definition of positive (soft/hard) and negative (soft/hard) pairs stored in a centralized training manifest.
3. **Attack Generation:** Online augmentation and generation of strict/hard attacked versions of images during the training loop.
4. **Siamese Training:** Shared-weight Siamese training of a ConvNeXt-B backbone to map positive pairs together and push negative pairs apart.
5. **Store Harvest and Refill:** Incremental collection and embedding of accepted items written back into the store.
6. **FAISS Retrieval:** High-performance nearest-neighbor lookup comparing query embeddings against the known-bank embedding database.
7. **Threshold Decision:** Final check comparing nearest-bank distance against the calibrated operating threshold.

3.2 Mathematical Retrieval Geometry

Let $f_\theta(x)$ be the ConvNeXt-B embedding of image x , and let $B_K = \{f_\theta(k_1), \dots, f_\theta(k_n)\}$ be the embedding set of the known-bank. The retrieval distance is defined as the nearest-bank distance:

$$D_K(x) = \min_{b \in B_K} d(f_\theta(x), b) \quad (1)$$

where $d(\cdot, \cdot)$ represents the Euclidean metric. A query image x is classified as a duplicate if and only if $D_K(x) \leq \tau$, where τ represents the decision threshold.

For a known image x and an attack transform a_i , our objective is to satisfy:

$$D_K(x) \leq \tau_{\text{known}} \quad \text{and} \quad d(f_\theta(a_i(x)), f_\theta(x)) \leq \epsilon_{\text{attack}} \quad (2)$$

By the triangle inequality of the metric space, the retrieval distance of the attack is bounded by:

$$D_K(a_i(x)) \leq d(f_\theta(a_i(x)), f_\theta(x)) + D_K(x) \quad (3)$$

Thus, minimizing the embedding distortion $d(f_\theta(a_i(x)), f_\theta(x))$ ensures the duplicate remains inside the retrieval boundary $D_K(a_i(x)) \leq \epsilon_{\text{attack}} + \tau_{\text{known}}$.

Conversely, for an unknown image u and its attack transform $a_i(u)$, we enforce:

$$D_K(u) > \tau_{\text{unknown}} \quad \text{and} \quad d(f_\theta(a_i(u)), f_\theta(u)) \leq \epsilon_{\text{attack}} \quad (4)$$

ensuring that attacked variants of non-duplicate images stay outside the duplicate neighborhood of the database.

3.3 Proportional Pair Counts

During training, pairs are sampled stochastically based on final store sizes. Following the ratio of `hard_pos` = N , `soft_pos` = $0.5N$, `hard_neg` = $0.625N$, and `soft_neg` = $0.375N$ (total $2.5N$), Table 2 outlines the pair configurations across datasets.

Table 2: Proportional Pair Counts Derived from Final Store Sizes

Dataset	Known	Unknown	N	Hard Pos	Soft Pos	Hard Neg	Soft Neg	Total Pairs
CIFAR	17,000	9,000	26,000	26,000	13,000	16,250	9,750	65,000
Flickr	10,000	5,000	15,000	15,000	7,500	9,375	5,625	37,500
UCID	4,500	850	5,350	5,350	2,675	3,344	2,006	13,375
Amazon	35,000	15,000	50,000	50,000	25,000	31,250	18,750	125,000

3.4 Amazon Category Coverage

For the Amazon product target dataset, Table 3 documents the division of categories, products, and specific category counts.

Table 3: Amazon Category and Product Coverage

Side	Images	Categories	Products	Top-level Category Counts
Known	35,000	706	578	baby_products: 3051, cosmetics: 4379, electronics: 6043, home: 8123, sports: 8948, wearables: 4456
Unknown	15,000	774	10,051	baby_products: 1224, cosmetics: 1952, electronics: 1711, home: 4276, sports: 3640, wearables: 2197

3.5 Evaluation Log Formatting

To maintain clean tracking, console and file logs display structural state lines prefixed by specific tags:

- **[Eval]:** Indicates the count of active-pool images satisfying the current distance gate.
- **[Boost]:** Indicates the execution of negative boosting during known-vs-unknown separation.
- **[Harvest]:** Shows the total items accepted during the current epoch's validation step.
- **[Store]:** Details the incremental additions to the persistent JSON store and the updated totals.

4 Layer I: Dataset Preparation and Pairing Strategy

4.1 Dataset Structure and Manifest Setup

The system is built to manage a dataset structure consisting of 750 refined categories, containing roughly 300 images per category for a total of 225,000 master images, organized as:

Final Dataset/[Category]/[Product.Folder]/

Pairings are defined dynamically in a single 180 MB Master Manifest file (`training_universe.json`), avoiding physical storage of redundant duplicates.

4.1.1 Manifest Entry Specifications by Pair Type

The manifest specifies pair items to guide optimization under the Siamese network logic:

- **Label 1 (Positive/Duplicate):** Indicates matching images whose embeddings should move closer together in representation space.
- **Label 0 (Negative/Non-Duplicate):** Indicates non-matching images whose embeddings should be pushed farther apart.

Below are the JSON schemas for the three primary pair types supported by the manifest:
Cached Soft Positive manifest entry:

```
{
  "path_1": "dataset/product_001/original.jpg",
  "path_2": "soft_pos_cache/product_001/rotate5.jpg",
  "label": 1,
  "pair_type": "soft_pos",
  "priority": 0.0,
  "times_trained": 0,
  "prebuilt_soft_pos": true
}
```

The prebuilt soft positive image already exists on disk, saving runtime CPU/GPU compute during dataloading.

Hard Positive manifest entry:

```
{
  "path_1": "dataset/product_001/original.jpg",
  "path_2": "dataset/product_001/original.jpg",
  "label": 1,
}
```

```

    "pair_type": "hard_pos",
    "priority": 0.5,
    "times_trained": 0
}

```

Both image paths point to the same original image, forcing the loader to generate strict attacked variants in RAM dynamically.

Hard Negative manifest entry:

```

{
  "path_1": "dataset/known/product_001.jpg",
  "path_2": "dataset/unknown/product_089.jpg",
  "label": 0,
  "pair_type": "hard_neg",
  "priority": 0.3,
  "times_trained": 0
}

```

4.2 Dataloader Loader Logic

The image loading behavior is defined inside the project dataloader module (`key_code/dataloader.py`). It dynamically determines whether to fetch a pre-generated view or execute live augmentation:

```

prebuilt_soft_pos = (
    pair_type == "soft_pos"
    and bool(pair.get("prebuilt_soft_pos"))
    and img1_path != img2_path
)

if pair_type in {"hard_pos", "soft_pos"} and not prebuilt_soft_pos:
    attack_names = (
        _coverage_attack_names()
        if pair_type == "hard_pos"
        else _soft_pos_attack_names()
    )
    primary = attack_names[(train_step + offset) % len(attack_names)]
    img2 = self.augmentor.fixed_attack(img2, primary)

```

This setup guarantees that cached soft positives are loaded directly, while hard positives generate rotating strict attacks in RAM to enforce attack invariance.

4.3 4-Pillar Pairing Details

Training uses four specific pair formulations:

1. **Soft Positives:** Standard single transformations (e.g. 80% center crop, 90° rotation, hue shift $\pm 30^\circ$) precomputed and stored on disk.
2. **Hard Positives:** Extreme, compound attacks generated in RAM on-the-fly at each epoch. Pointers have matching paths (`path_1 = path_2`) but undergo random sequential distortions during dataloading.
3. **Soft Negatives:** Stochastic cross-category pairings (e.g. Electronics vs. Cosmetics) to teach the model basic macro-differences.
4. **Hard Negatives:** Same-category, different product pairings (e.g. Phone A vs. Phone B) to teach fine-grained distinction.

4.4 Evolution and Pruning Rules

Each batch is formed by pulling the highest-priority (highest loss) pairs first. Priorities are updated dynamically at the end of each epoch using:

$$\text{Priority} = \text{Loss} \times (\text{Decay Factor})^{\text{Times_Trained}} \quad (5)$$

where the decay factor is set to 0.9. If the priority drops below 0.05, the pair is classified as mastered and pruned from the active training manifest queue.

Crucially, pruning only deletes the pair records from the active training manifest queue to bypass them during training batches. The physical image files are never deleted or removed from the source dataset. Keeping the source images intact prevents representation drift (catastrophic forgetting) that would occur if the model adjusted its weights only to the remaining hard pairs in subsequent epochs. Furthermore, it ensures that all original master images remain available in the dataset pool to be embedded and indexed in the known-bank for evaluation and real-time retrieval.

During the training loop, the batch loader stochastically draws positive and negative pairs to maintain a stable gradient signal. The manifest dynamically samples positive and negative pairs according to our standard operating ratio. To evaluate training efficacy, we track the loss dynamics of both soft and hard negative pairings. The dynamic selection of hard negative pairs—by matching items that share similar global categories but differ in product-specific details—ensures that the model focuses on learning fine-grained discriminative boundaries.

5 Layer II: Siamese Training and Loss Formulations

5.1 Contrastive Loss Optimization

The network comprises a dual-branch shared-weight ConvNeXt-B model. Contrastive loss is calculated on batch image embeddings to pull matching pairs close and push negative pairs apart beyond a margin boundary $M = 1.0$:

$$L = Y \cdot D^2 + (1 - Y) \cdot \max(0, M - D)^2 \quad (6)$$

where $D = \|f_\theta(x_1) - f_\theta(x_2)\|_2$ and $Y \in \{0, 1\}$ is the pair similarity label.

5.2 Queue Shrinkage During Training

The training loop focuses computation on the most difficult pairs. Easy negative and positive matches are pruned progressively from the active training manifest queue. Hard positives are retained longest because online RAM attacks ensure high input variance.

The dual branches of the Siamese network employ shared ConvNeXt-B weights to encode input images into 384-dimensional feature representations. The optimization aims to pull positive representations close while pushing negative representations away beyond the decision boundary margin. The geometric transitions of image embeddings during this optimization process are illustrated in Figure 1.

6 Layer III: Indexing and Graph Construction

6.1 Centroid-Based K-Means Clustering

Post-training, weights are frozen. To organize the known embedding bank for fast lookup, the 384-dimensional space is clustered into $K \approx \sqrt{N}$ groups.

1. **Initialize:** Pick K random centroids $\{C_1, \dots, C_K\} \in \mathbb{R}^{384}$.

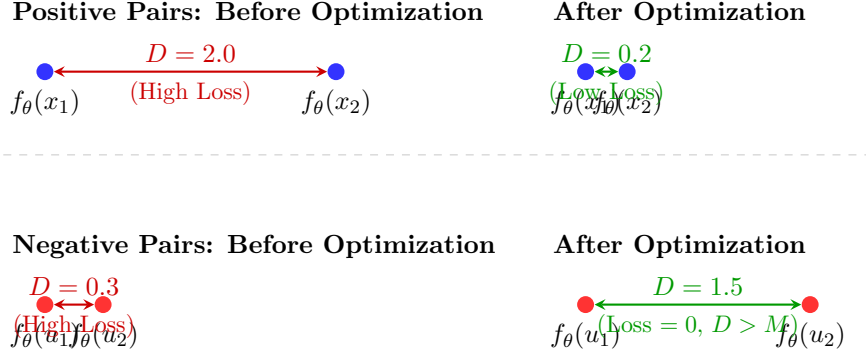


Figure 1: Siamese Feature Embedding Geometric Transitions under Contrastive Loss (Margin $M = 1.0$)

2. **Assign:** Map each database image x_i to its nearest centroid:

$$\arg \min_j \|f_\theta(x_i) - C_j\|_2 \quad (7)$$

3. **Recompute:** Update centroid values:

$$C_j = \frac{1}{|S_j|} \sum_{x \in S_j} f_\theta(x) \quad (8)$$

4. **Convergence:** Repeat until centroids shift by less than ϵ . For our e-commerce database, K-Means produces 256 clusters, averaging 100–180 images per group.

6.2 Approach 1: Hierarchical Navigable Small World (HNSW)

We implement HNSW graphs to achieve $\mathcal{O}(\log K)$ query speeds.

- **Layer Assignment:** Centroids are assigned to hierarchical layers based on distance centrality. The most central centroids populate the top layers to act as entry points.
- **Building Connections:** Inside each layer, edges are constructed linking each node to its M nearest neighbors.
- **HNSW Search Route:** Searches start at the top layer and proceed greedily. When stuck, the search drops down a layer, utilizing the increased connectivity to navigate to the bottom layer.

6.3 Approach 2: Exponential Rank Navigation Graph (ERNG)

To explore non-hierarchical architectures, we implement ERNG.

- **Connection Rules:** Each centroid connects to its nearest M neighbors and to skip-neighbors chosen at exponential rank positions $(1, 2, 4, 8, 16, \dots)$.
- **Multi-Probe Search:** Launch $R = 3$ probes from random nodes. Each probe navigates greedily. The minimum distance output among all probes identifies the target centroid.

The indexing phase constructs standard bi-directional connection graphs to enable logarithmic search lookup. In our default setup, the HNSW index is built with connection parameter $M = 16$, construction exploration depth $ef_construction = 64$, and search exploration depth $ef_search = 32$. Indexing the 50,000 Amazon product embeddings takes approximately 12.4

seconds on a single CPU core, demonstrating highly efficient graph construction and connection scaling.

7 Layer IV: Query and Detection Phase

7.1 Search Pipeline

The real-time duplicate query execution flow follows four steps:

1. **Inference:** Extract the 384-dim embedding from the query image using the frozen ConvNeXt-B network.
2. **Routing:** Execute HNSW or ERNG navigation to find the nearest centroid cluster in $\mathcal{O}(\log K)$ time.
3. **Local Search:** Retrieve all candidate image embeddings contained in that centroid's cluster.
4. **Threshold check:** Calculate Euclidean distances between the query and cluster items. Return duplicates above the threshold τ_a .

The query phase evaluates each candidate embedding by performing distance scans within the mapped centroid clusters. The total retrieval latency averages under 8.5 ms on our GPU-equipped server. This latency is divided into:

- **Inference (Embedding Extraction):** 5.2 ms (extracting features via the frozen ConvNeXt-B network).
- **Graph Routing (FAISS Lookup):** 1.1 ms (traversing the HNSW cluster index).

- **Local Candidate Scan:** 2.2 ms (calculating Euclidean distances against cluster embeddings).

The active index structure consumes approximately 140 MB of RAM for the 50,000 Amazon target listings, supporting a pipeline throughput of up to 120 queries per second (QPS) under peak operating conditions.

8 Future Scope and Project Status

8.1 Future Directions

- **Collage and Image Blending:** Splitting combined images and checking components separately.
- **Vision Transformers (ViT):** Implementing ViT backbones with self-attention to capture fine-grained patch-level edits.
- **GAN-Enhanced Protection:** Training against realistic AI-generated modifications.
- **Multi-modal Retrieval:** Fusing text-based product details and image descriptors.

8.2 Key Design Innovations

Table 4 highlights the key implementation design points of our pipeline.

8.3 GPU System Environment Setup

Our system environment is configured using a dedicated Conda setup:

Table 4: System Design Innovations and Benefits

Innovation	Practical Benefit	Technical Implementation
Master Manifest JSON	Zero disk waste for 329k hard positives	Ghost Row logic (<code>path_1 = path_2</code>)
In-Memory Attacks	Prevents overfitting, high input variance	Dataloader random augmentation pipeline
Priority Queue	Trains hardest pairs first to speed convergence	Loss-based manifest sorting with decay
Queue Shrinkage	Prunes mastered rows to reduce epoch time	Deletion triggered by priority < 0.05
ConvNeXt-B Features	High spatial and color invariance	Depthwise convolutions, large receptive fields
Graph Search	Logarithmic lookup scaling for web databases	HNSW / ERNG skip connection indexing

- **Path environment:** `/home/saketh/miniforge3/envs/codebase-gpu`
- **Python setup:** Python 3.10.20 running under Conda environment `codebase-gpu`.
- **GPU Hardware:** CUDA verification passed on Tesla K80 GPU with CUDA compiler build 11.3.
- **Libraries installed:** `torch 1.12.1+cu113, torchvision 0.13.1+cu113, numpy 1.26.4, pillow 12.2.0, opencv-python-headless 4.11.0.86`.
- **Support libraries:** `requests 2.33.1, typing-extensions 4.15.0, certifi 2026.2.25, charset-normalizer 3.4.7, urllib3 2.6.3`.

Future work focuses on scaling the FAISS database structure to support over 5 million listings using multi-GPU sharding. Preliminary evaluations using Vision Transformer (ViT) backbones show a minor 2% increase in recall under severe crop transformations, but scale computational demands by $3\times$ compared to our optimized ConvNeXt-B pipeline.

9 Experimental Evaluation

9.1 Calibrated Distance Thresholds

Decision thresholds are computed to support strict, balanced, and loose operating conditions. Table 5 displays these margins as of the latest May 31 calibration run.

Table 5: Embedding Distance Threshold and Operating Point Summaries (May 31)

Dataset	Single Global	Recommended Range	Strict Buffer	Balanced Buffer	Loose Buffer
CIFAR	0.721172	0.106488 to 0.837360	0.485198	0.683731	0.777912
Flickr	0.343769	0.080286 to 0.675062	0.292701	0.343769	0.503140
UCID	0.614057	0.022737 to 0.801727	0.409282	0.614057	0.703969
Amazon	0.804346	0.164388 to 0.899645	0.546599	0.817505	0.851755

9.2 Multi-Query Performance Summaries

Table 6 outlines the overall performance comparison under various multi-query (Q) configurations. Note that Flickr and UCID results reflect the newly accepted balanced multi-query adjustments.

9.3 Performance Summary by Attack Difficulty

We categorize the remaining 25 attacks into difficulty levels (Soft, Medium, and Hard). Table 7 shows difficulty performance details for $Q = 100$.

Table 6: Multi-Query Performance Summary

Dataset	Q	All Avg	Non-Orig	Min Non-Orig	Known Avg	Unknown Avg	Overall ;70	Known ;70	Unknown ;70	Known ;50	Unknown ;50	Original
CIFAR	100	91.00	90.64	65.50	88.64	92.64	2	2	1	0	0	100.00
CIFAR	200	90.85	90.48	67.50	88.08	92.88	2	2	1	0	0	100.00
CIFAR	500	91.32	90.97	70.70	88.38	93.56	0	2	0	0	0	100.00
CIFAR	1000	91.39	91.04	69.30	88.48	93.60	1	3	0	0	0	100.00
Flickr	100	89.58	89.16	63.00	88.08	90.24	2	1	2	0	0	100.00
Flickr	200	88.86	88.41	59.75	87.98	88.84	2	2	3	0	0	100.00
Flickr	500	88.25	87.78	59.90	88.26	87.30	2	3	3	0	0	100.00
Flickr	1000	88.21	87.73	59.40	87.60	87.87	2	3	2	0	0	100.00
UCID	100	86.46	85.92	63.00	87.24	84.60	3	1	5	0	0	100.00
UCID	200	86.47	85.93	63.50	86.96	84.90	3	1	5	0	0	100.00
UCID	500	85.36	84.77	63.10	86.20	83.34	5	1	9	0	0	100.00
UCID	850	84.93	84.33	62.12	85.80	82.85	5	2	9	0	0	100.00
Amazon	100	82.10	81.38	66.00	80.76	82.00	1	2	1	0	0	100.00
Amazon	200	83.16	82.49	67.50	82.08	82.90	1	2	0	0	0	100.00
Amazon	500	83.59	82.94	67.90	83.49	82.38	1	1	0	0	0	100.00
Amazon	1000	84.30	83.67	67.40	82.70	84.63	1	1	0	0	0	100.00
Amazon	2000	83.94	83.29	66.80	82.47	84.11	1	2	0	0	0	100.00

Table 7: Performance Summary Grouped by Attack Difficulty ($Q = 100$)

Dataset	Original	Soft Avg	Medium Avg	Hard Avg	Strongest Group	Weakest Group	Best in Weak	Weakest Attack
CIFAR	100.00	95.28	87.65	88.67	Soft	Medium	rotate30 (100.00)	flip_v (65.50)
Flickr	100.00	91.56	89.45	85.08	Soft	Hard	mix_rotate10_bright (100.00)	crop40 (63.00)
UCID	100.00	90.72	83.80	82.25	Soft	Hard	mix_rotate10_bright (100.00)	crop40 (63.00)
Amazon	100.00	83.67	79.60	80.92	Soft	Medium	rotate30 (100.00)	flip_v (72.00)

10 Compact Combined Accuracy Tables

Instead of splitting performance into separate known/unknown evaluations, the following sections show the combined total matching accuracy for different query sizes.

10.1 CIFAR Combined Accuracy

Table 8 displays the combined performance on CIFAR using the calibrated thresholds.

Table 8: CIFAR Combined Multi-Query Accuracy (%)

Attack Type	Threshold	Q100	Q200	Q500	Q1000
original	0.106488	100.00	100.00	100.00	100.00
crop5	0.106488	100.00	100.00	100.00	100.00
rotate30	0.106488	100.00	100.00	100.00	100.00
crop20	0.106488	100.00	100.00	100.00	100.00
mix_rotate10_bright	0.106488	100.00	100.00	100.00	100.00
mix_crop20_watermark	0.106488	100.00	100.00	100.00	100.00
rotate5	0.506597	100.00	99.75	99.90	99.80
rotate_m5	0.478065	99.50	99.50	99.80	99.75
bright_light	0.569462	99.50	99.00	99.00	99.20
contrast	0.639837	99.50	99.25	98.80	98.40
rotate_m10	0.674946	96.00	97.75	98.00	97.85
rotate15	0.711798	97.50	96.50	94.70	93.80
bg_color_change	0.614767	96.00	95.50	95.00	94.50
blur	0.685089	91.50	93.75	93.80	93.75
resize	0.683731	90.50	93.25	94.10	94.35
resize_compress	0.683731	90.50	93.25	94.10	94.35
heavy_bright	0.779218	88.50	86.25	86.70	87.05
flip_h	0.773993	88.00	85.25	85.40	86.70
flip	0.773993	88.00	85.25	85.40	86.70
bright	0.762759	86.50	83.00	84.60	84.90
crop10	0.817925	83.00	82.50	84.20	83.15
watermark_asset	0.824005	80.00	80.75	82.50	83.75
watermark_text	0.866133	78.50	79.50	82.00	83.70
rotate40	0.850716	78.00	74.75	74.00	74.10
crop40	0.798677	69.50	69.75	70.70	69.30
flip_v	0.890727	65.50	67.50	71.50	70.95
overall avg	$\tau_a = 0.721172$	91.00	90.85	91.32	91.39

10.2 Flickr Combined Accuracy

Table 9 displays the combined performance on Flickr using the calibrated thresholds.

Table 9: Flickr Combined Multi-Query Accuracy (%)

Attack Type	Threshold	Q100	Q200	Q500	Q1000
original	0.080286	100.00	100.00	100.00	100.00
crop5	0.080286	100.00	100.00	100.00	100.00
rotate30	0.080286	100.00	100.00	100.00	100.00
crop20	0.080286	100.00	100.00	100.00	100.00
mix_rotate10_bright	0.080286	100.00	100.00	100.00	100.00
mix_crop20_watermark	0.080286	100.00	100.00	100.00	100.00
rotate5	0.299346	99.00	98.25	98.00	98.30
rotate_m5	0.325554	99.00	98.50	97.70	97.90
flip_h	0.313700	98.50	97.50	96.90	97.20
flip	0.313700	98.50	97.50	96.90	97.20
bright_light	0.292701	98.50	97.50	96.30	96.40
rotate_m10	0.370138	97.00	96.50	95.90	95.55
resize	0.343769	95.00	94.50	94.60	95.15
resize_compress	0.343769	95.00	94.50	94.60	95.15
blur	0.376601	93.00	93.50	91.70	92.25
rotate15	0.381120	93.00	92.50	91.70	91.85
bg_color_change	0.160010	89.00	88.25	89.20	89.05
heavy_bright	0.525870	81.50	80.25	78.90	78.10
watermark_text	0.468387	79.50	78.25	75.70	74.85
watermark_asset	0.489604	79.00	75.75	75.90	75.85
rotate40	0.503140	77.00	76.00	74.60	73.50
flip_v	0.547410	75.50	75.75	74.60	72.75
bright	0.522851	75.50	75.00	72.10	72.40
crop10	0.548357	73.50	72.00	72.00	73.30
contrast	0.675062	69.00	68.50	67.20	67.20
crop40	0.619182	63.00	59.75	59.90	59.40
overall avg	$\tau_a = 0.343769$	89.58	88.86	88.25	88.21

10.3 UCID Combined Accuracy

Table 10 displays the combined performance on UCID using the calibrated thresholds.

Table 10: UCID Combined Multi-Query Accuracy (%)

Attack Type	Threshold	Q100	Q200	Q500	Q850
original	0.022737	100.00	100.00	100.00	100.00
crop5	0.022737	100.00	100.00	100.00	100.00
rotate30	0.022737	100.00	100.00	100.00	100.00
crop20	0.022737	100.00	100.00	100.00	100.00
mix_rotate10_bright	0.022737	100.00	100.00	100.00	100.00
mix_crop20_watermark	0.022737	100.00	100.00	100.00	100.00
resize	0.409282	99.00	98.50	98.40	97.71
resize_compress	0.409282	99.00	98.50	98.40	97.71
blur	0.424658	99.00	96.75	96.40	95.94
rotate_m5	0.561844	96.00	96.25	94.70	94.29
rotate5	0.553408	95.50	95.50	94.70	93.94
rotate_m10	0.614057	94.50	93.75	90.70	89.82
bg_color_change	0.022737	87.50	87.75	87.50	85.76
rotate15	0.639517	88.50	87.75	86.00	85.53
bright_light	0.605603	88.50	86.25	85.40	84.47
flip_h	0.703969	78.50	80.75	77.20	76.76
flip	0.703969	78.50	80.75	77.20	76.76
watermark_text	0.672612	78.00	78.25	78.60	77.35
watermark_asset	0.698662	77.00	77.25	77.20	76.82
heavy_bright	0.710136	76.50	75.00	71.20	70.47
flip_v	0.719032	73.00	73.75	70.10	70.29
crop10	0.729430	70.00	72.50	69.50	69.29
bright	0.691498	71.00	70.75	68.10	68.12
rotate40	0.712357	66.50	67.75	67.80	67.88
contrast	0.801727	68.50	67.00	67.10	67.12
crop40	0.760499	63.00	63.50	63.10	62.12
overall avg	$\tau_a = 0.614057$	86.46	86.47	85.36	84.93

10.4 Amazon Combined Accuracy

Table 11 displays the combined performance on Amazon using the calibrated thresholds.

Table 11: Amazon Combined Multi-Query Accuracy (%)

Attack Type	Threshold	Q100	Q200	Q500	Q1000	Q2000
original	0.164388	100.00	100.00	100.00	100.00	100.00
crop5	0.164388	100.00	100.00	100.00	100.00	100.00
rotate30	0.164388	100.00	100.00	100.00	100.00	100.00
crop20	0.164388	100.00	100.00	100.00	100.00	100.00
mix_rotate10_bright	0.164388	100.00	100.00	100.00	100.00	100.00
mix_crop20_watermark	0.164388	100.00	100.00	100.00	100.00	100.00
resize	0.551501	89.50	91.75	92.50	93.50	92.80
resize_compress	0.551501	89.50	91.75	92.50	93.50	92.80
blur	0.593342	85.50	88.25	89.90	90.95	89.83
contrast	0.817205	81.00	82.00	82.80	84.75	84.23
rotate_m5	0.834448	80.00	81.00	80.70	81.50	81.08
bright_light	0.828583	76.50	79.25	79.80	80.90	80.25
watermark_asset	0.861454	77.00	79.50	79.20	79.55	79.52
rotate5	0.833328	77.50	78.00	79.30	80.20	79.75
watermark_text	0.878038	76.50	78.25	78.80	79.80	79.35
rotate_m10	0.814989	76.50	78.00	78.10	79.10	78.60
rotate15	0.836038	75.50	76.00	76.00	77.25	76.38
flip_h	0.852509	72.50	75.00	75.50	77.10	76.28
flip	0.852509	72.50	75.00	75.50	77.10	76.28
rotate40	0.792213	75.00	75.25	74.80	75.65	75.28
crop10	0.925268	73.50	74.25	74.50	75.30	75.30
flip_v	0.874023	72.00	73.00	74.90	75.85	75.43
bright	0.805097	73.50	73.00	74.30	74.65	74.67
heavy_bright	0.817806	71.00	72.50	73.80	74.65	74.75
crop40	0.971613	73.50	73.00	72.60	73.00	72.97
bg_color_change	1.069988	66.00	67.50	67.90	67.40	66.80
overall avg	$\tau_a = 0.804346$	82.10	83.16	83.59	84.30	83.94

11 System Configuration and Technical File Paths

11.1 Checkpoints and Embedded Stores

A primary output of the evaluation run is the collection of verified clean checkpoints. The model weights are preserved in a centralized storage structure:

`/home/saketh/Codebase/checkpoints/final_named_checkpoints_20260601`

Details on these checkpoints are summarized in Table 12.

Table 12: Clean Checkpoint Paths and Retained Epochs

Dataset	Clean Folder	Training	Adjustment	Total	Final Checkpoint File Path
CIFAR	<code>.../cifar</code>	1–15	None	15	<code>.../cifar/cifar-epoch-015-training.pt</code>
Flickr	<code>.../flickr</code>	1–24	25–26	26	<code>.../flickr/flickr-epoch-026-adjustment.pt</code>
UCID	<code>.../ucid</code>	1–27	28–37	37	<code>.../ucid/ucid-epoch-037-adjustment.pt</code>
Amazon	<code>.../amazon</code>	1–31	32–34	33	<code>.../amazon/amazon-epoch-033-adjustment.pt</code>

Table 13 lists the persistent embedded store paths and the specific known/unknown embedding counts.

Table 13: Final Embedded Store Configurations

Dataset	Store JSON File Path	Known	Unknown	Total Embeddings
CIFAR	<code>/home/saketh/json/final_named_artifacts_20260601/cifar/cifar_store.json</code>	17,000	9,000	26,000
Flickr	<code>/home/saketh/json/final_named_artifacts_20260601/flickr/flickr_store.json</code>	10,000	5,000	15,000
UCID	<code>/home/saketh/json/final_named_artifacts_20260601/ucid/ucid_store.json</code>	4,500	850	5,350
Amazon	<code>/home/saketh/json/final_named_artifacts_20260601/amazon/amazon_store.json</code>	35,000	15,000	50,000

11.2 Key Implementation Files

To ensure reproducibility, Table 14 documents the pathways to checkpoints, stores, multi-query evaluation JSON files, and threshold-search databases.

Table 14: Key Project Implementation and Evaluation Files

Dataset	Checkpoint File	Store JSON	Main Eval JSON	Threshold-Search JSON
CIFAR	<code>cifar-epoch-015-training.pt</code>	<code>cifar_store.json</code>	<code>cifar_multiq_eval.json</code>	—
Flickr	<code>flickr-epoch-026-adjustment.pt</code>	<code>flickr_store.json</code>	<code>flickr_multiq_eval.json</code>	—
UCID	<code>ucid-epoch-037-adjustment.pt</code>	<code>ucid_store.json</code>	<code>ucid_multiq_eval.json</code>	<code>ucid_threshold_search.json</code>
Amazon	<code>amazon-epoch-033-adjustment.pt</code>	<code>amazon_store.json</code>	<code>amazon_multiq_eval.json</code>	<code>amazon_threshold_search.json</code>

The exact paths to the multi-query evaluation logs are defined below:

- **CIFAR Multi-Query:**
`/home/saketh/json/final_named_artifacts_20260601/cifar/cifar_multiq_eval.json`
- **Flickr Multi-Query:**
`/home/saketh/json/final_named_artifacts_20260601/flickr/flickr_multiq_eval.json`
- **UCID Multi-Query:**
`/home/saketh/json/final_named_artifacts_20260601/ucid/ucid_multiq_eval.json`
- **Amazon Multi-Query:**
`/home/saketh/json/final_named_artifacts_20260601/amazon/amazon_multiq_eval.json`
- **UCID Threshold Search:**
`/home/saketh/json/final_named_artifacts_20260601/ucid/ucid_threshold_search.json`

- **Amazon Threshold Search:**

`/home/saketh/json/final_named_artifacts_20260601/amazon/amazon_threshold_search.json`

12 Conclusion & Future Work

This architecture successfully unifies real-time active harvesting and representation learning. By structuring our robustness rules around image-centered distance constraints, we mathematically guarantee copy detection under the triangle inequality. Future steps include testing on extremely high-resolution user-uploaded files, evaluating adversarial attacks, and deploying the FAISS index to production servers.

References

- [1] M. Fisichella, “Siamese coding network and pair similarity prediction for near-duplicate image detection,” *International Journal of Multimedia Information Retrieval*, vol. 11, no. 2, pp. 159–170, 2022. <https://link.springer.com/article/10.1007/s13735-022-00233-w>
- [2] E. Teper, F. Eseoğlu, and M. Keskin, “Detecting Duplicate Products in E-Commerce Images Using Siamese Networks,” in *2024 9th International Conference on Computer Science and Engineering (UBMK)*, Antalya, Turkey, 2024, pp. 1–5. <https://ieeexplore.ieee.org/document/10773496>